



BASES DE DATOS AVANZADAS

Classroom: Bases de Datos Avanzadas



12 DE MAYO DE 2026

carlos.herrera@upa.edu.mx

<https://github.com/ISC-UPA/2026-2-TIID5-DB2>

Contenido

- 1. S01..... 2
 - 1.1. Referencias 2
 - 1.2. Technological Requirements 2
 - 1.1. Relaciones..... 3
- 2. S02..... 6
- 3. S03..... 7
- 4. S04..... 10
- 5. S05..... 11
- 6. S06..... 11
- 7. S07 12
- 8. S08..... 13
- 9. S09..... 14

1. S01

1.1. Referencias

<https://www.w3schools.com/mysql>

<https://conclase.net/mysql>

1.2. Technological Requirements

VS Code <https://code.visualstudio.com/Download>

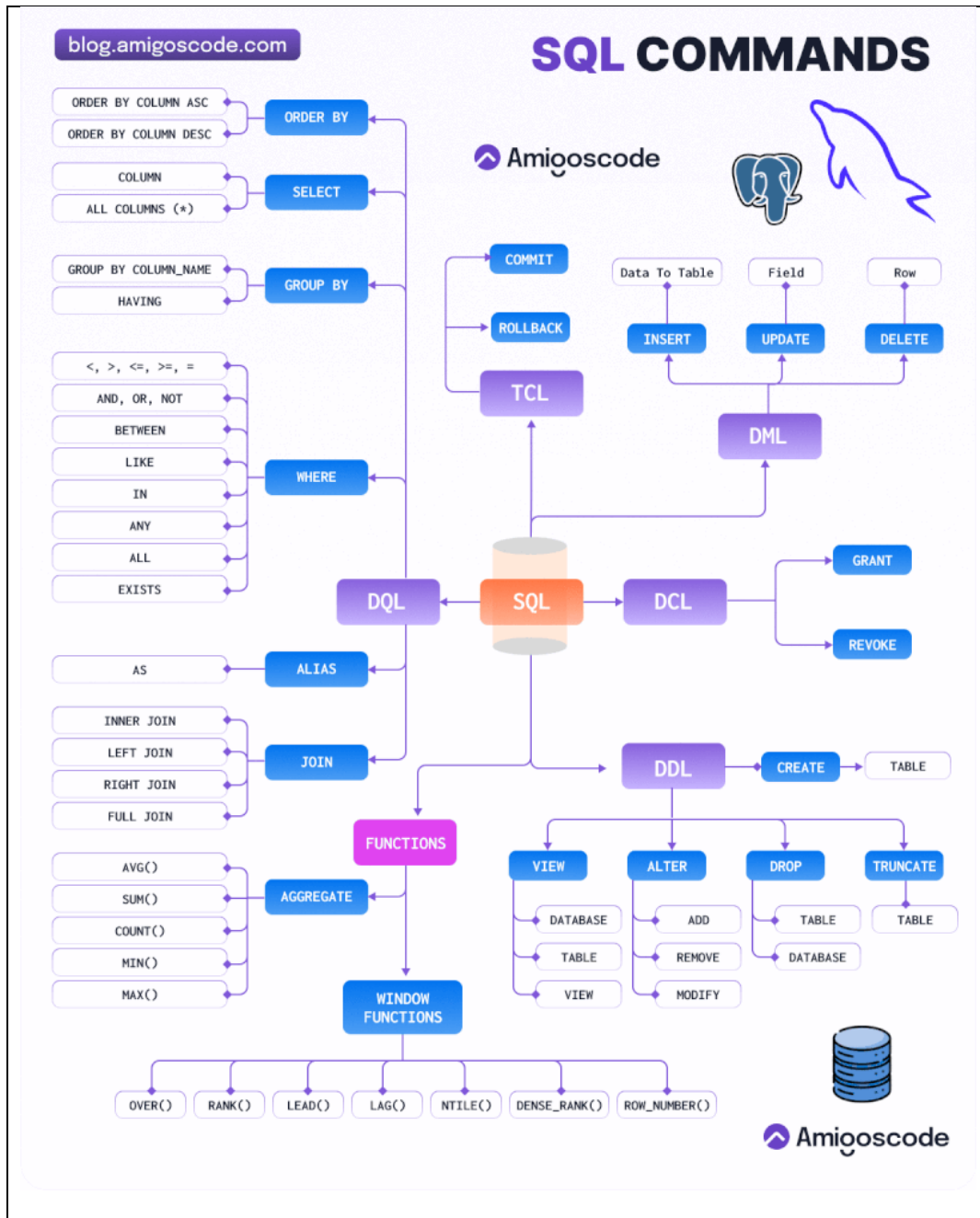
Extensions: **Database Client**

Extensions: **MongoDB for VS Code**

XAMPP

MongoDB

PowerBI Desktop



SQL Statements

CREATE
ALTER
DROP
RENAME
TRUNCATE
COMMENT

Data definition language (DDL)

SELECT
INSERT
UPDATE
DELETE
MERGE

Data manipulation language (DML)

GRANT
REVOKE

Data control language (DCL)

COMMIT
ROLLBACK
SAVEPOINT

Transaction control language (TCL)

¿CÓMO FUNCIONAN LOS ÍNDICES EN SQL?

"Un índice puede reducir consultas de segundos a milisegundos"

¿QUÉ ES UN ÍNDICE?

Un índice es una estructura que permite encontrar datos en una tabla sin necesidad de recorrerla completa. Funciona como el índice de un libro.

Tabla (Datos) → Índice (Atajos)

```
CREATE INDEX idx_nombre ON usuarios(nombre);
```

¿POR QUÉ USAR ÍNDICES?

- Búsquedas rápidas: Aceleran las consultas, especialmente en tablas grandes.
- Mejor rendimiento: Reduce el tiempo de respuesta y la carga del servidor.
- Optimización de consultas: El optimizador elige automáticamente el mejor índice disponible.
- Escalabilidad: Permite que tu base de datos crezca sin perder rendimiento.
- Mejor experiencia de usuario: Consultas más rápidas = apps más rápidas.

SIN ÍNDICE (FULL TABLE SCAN)

La base de datos revisa fila por fila hasta encontrar el dato buscado.

CON ÍNDICE (INDEX SCAN)

El índice va directo al lugar correcto sin revisar todo.

¿CÓMO FUNCIONA?

- Se crea un índice en una o más columnas.
- La base de datos construye una estructura denominada B-Tree.
- Cuando hace una consulta, el optimizador decide si usa el índice.
- Si usa el índice, va directamente a los datos relevantes (Index Scan).
- Si no, recorre toda la tabla (Full Table Scan).

Los datos están ordenados para permitir búsquedas rápidas.

EJEMPLO PRÁCTICO

Tabla de usuarios:

```
CREATE TABLE usuarios (
  id INT(11) UNSIGNED AUTO-
  INCREMENT PRIMARY KEY,
  nombre VARCHAR(50),
  email VARCHAR(100),
  edad INT
);
```

Consulta que puede usar el índice:

```
SELECT * FROM usuarios WHERE email = 'usuario1.com';
```

Usa el Índice Scan (Índice)

Explicación de la consulta:

```
Explicación de la consulta:
SELECT * FROM usuarios WHERE email = 'usuario1.com';
Rows Examined (Full Scan): 100000
Rows in result set: 1
Planning Time: 0.000 ms
Execution Time: 2.345 ms
```

Usa el Índice Scan (Índice)

Explicación de la consulta:

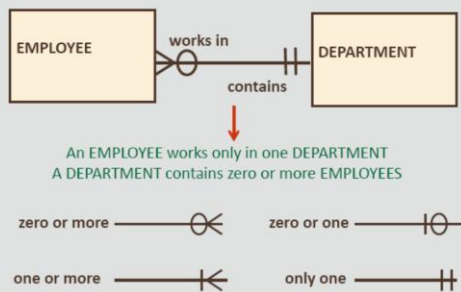
```
Explicación de la consulta:
SELECT * FROM usuarios WHERE email = 'usuario1.com';
Index Scan using idx_email on usuarios
Index Cond: (email = 'usuario1.com')
Planning Time: 0.338 ms
Execution Time: 2.345 ms
```

EN RESUMEN:

- Los índices son atajos que aceleran las consultas.
- Permiten búsquedas rápidas usando estructuras como B-Trees.
- Mejoran el rendimiento y la escalabilidad de tu base de datos.
- Úsalos con estrategia para obtener el máximo beneficio.

1.1. Relaciones

Information Engineering Notation



Data Model Notations

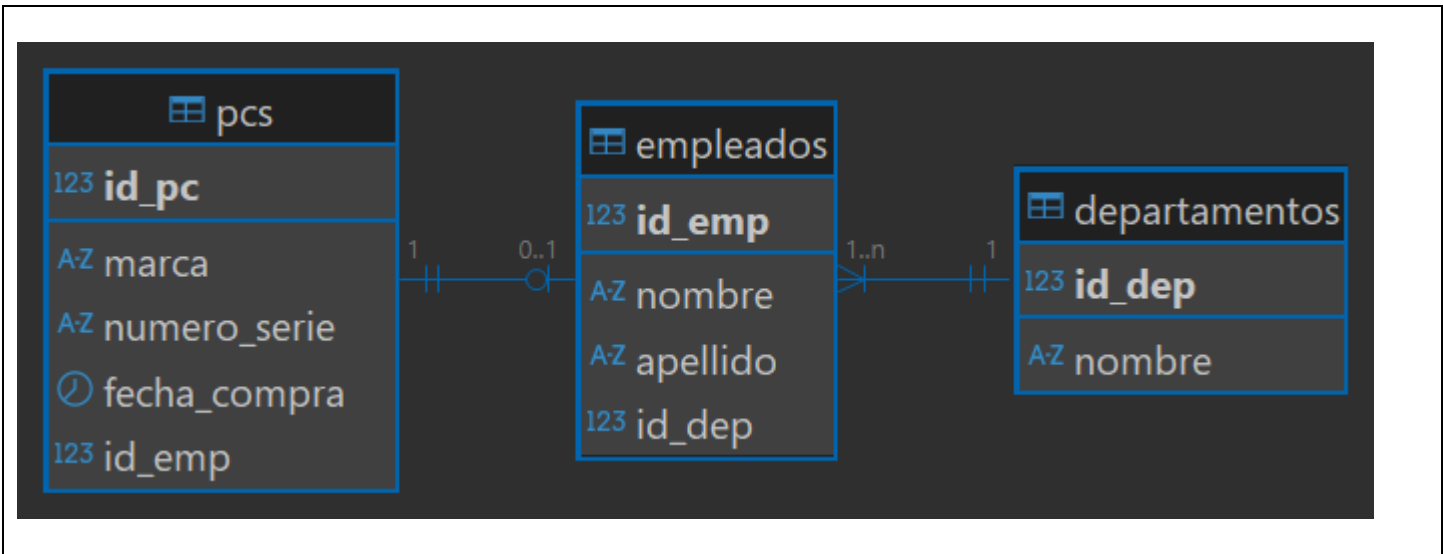
Notation (Read left to right)	Barker Notation	Bachman Notation	Information Engineering
Zero or one	— 0 —> — 0 —>	— 0 —> — 0 —>	— 0 —> — 0 —>
Only one	— 1 —> — 1 —>	— 1 —> — 1 —>	— 1 —> — 1 —>
Zero or more	— 0 —> — 0 —>	— 0 —> — 0 —>	— 0 —> — 0 —>
One or more	— 1 —> — 1 —>	— 1 —> — 1 —>	— 1 —> — 1 —>
Primary Key/Unique key	#	P	

Note: Barker notation is used for this course

Normalization: Is the process of organizing the attributes and tables of a relational database to minimize redundancy. No duplicate content, Increase the integrity of data.

Rule	Description
First Normal Form (1NF)	All attributes must be single-valued. (no multi-valued = atomic)
	1NF : If an attribute is multi-valued, create an additional entity and relate it to the original entity with a 1:M
Second Normal Form (2NF)	An attribute must be dependent on its entity's entire UID.
	No existen dependencias parciales
Third Normal Form (3NF)	No non-UID attributes can be dependent on another non-UID attribute.
	Ningún atributo no UID puede depender de otro atributo no UID.
	No existen dependencias transitivas.





```

-- Crear tabla de departamentos
CREATE TABLE departamentos (
    id_dep INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(50) UNIQUE NOT NULL
);

-- Crear tabla de empleados, debe pertenecer a un departamento
CREATE TABLE empleados (
    id_emp INT AUTO_INCREMENT,
    nombre VARCHAR(50),
    apellido VARCHAR(50) NOT NULL,
    id_dep INT not null,
    CONSTRAINT pk_empleados PRIMARY KEY (id_emp),
    FOREIGN KEY (id_dep) REFERENCES departamentos(id_dep)
);

-- Crear una tabla de pcs,
-- cada pc puede estar asignada a un empleado
-- cada empleado debe tener una pc asignada.
CREATE TABLE pcs (
    id_pc INT PRIMARY KEY AUTO_INCREMENT,
    marca VARCHAR(50),
    numero_serie VARCHAR(20) NOT NULL,
    fecha_compra DATE,
    id_emp INT,
    UNIQUE INDEX unique_serie (numero_serie ASC) VISIBLE
);

create unique index unique_emp on pcs(id_emp);          -- create or alter
ALTER TABLE pcs ADD CONSTRAINT unique_emp unique index (id_emp);

alter table pcs
add constraint fk_pcs_emp foreign key (id_emp) references empleados(id_emp);

-- Insertar datos en la tabla de departamentos
INSERT INTO departamentos (nombre) VALUES ('Ventas');
INSERT INTO departamentos VALUES (null, 'Informatica');
INSERT INTO departamentos VALUES (null, 'Recursos Humanos');

-- Insertar datos en la tabla de empleados
INSERT INTO empleados (nombre, apellido, id_dep) VALUES ('Jesus', 'Pérez', 1);
INSERT INTO empleados VALUES (null, 'María', 'Gómez', 2);
INSERT INTO empleados VALUES (null, 'Jose', 'Franco', 2);

-- Insertar datos en la tabla de pcs
INSERT INTO pcs (marca, numero_serie, fecha_compra, id_emp) VALUES ('Dell', 'SN12345', '2022-01-15', 1);
INSERT INTO pcs VALUES (null, 'HP', 'SN54321', '2023-03-10', 2);
INSERT INTO pcs VALUES (null, 'ASUS', 'SN67890', '2024-10-15', null); -- PC sin asignar

UPDATE pcs SET id_emp = 2 WHERE id_pc = 1;
-- Un Empleado una PC

```



Funciones: `select salary, codigo(salary) from employees;`
Procedimientos: `exec sp_calcularImpuesto @deptoid = 101;`

2. S02

Select fields
from tables
where filters logical operators: not and or
relational operators: >, <, =, >=, =, <>, !=
some words: like, between, in, is null, is not null
group by functions: min, max, count, sum, avg, std
having filters with functions
order by fields [desc]

Pattern Matching: LIKE Operator

- % denotes zero or more characters
- _ denotes one character

like 'A%' '_o%'

-- JOIN

SELECT table1.column, table2.column
FROM table1

[NATURAL JOIN table2] |
[JOIN table2 USING (column_name)] |
[inner JOIN table2
ON (table1.column_name = table2.column_name)] |
[LEFT|RIGHT|FULL OUTER JOIN table2
ON (table1.column_name = table2.column_name)] |
[CROSS JOIN table2];

→

3. S03

Funciones Escalares: String, Numeric, Date and Advanced (Flow Control)

https://www.w3schools.com/mysql/mysql_ref_functions.asp

<https://conclase.net/mysql/curso/cap11>

[MySQL :: MySQL 8.0 Reference Manual :: 14 Functions and Operators](#)

mariadb.com/docs/

Funciones de Cadena	Resultado	MySQL/Oracle
select ascii("A") Numero, char(65) as Caracter;	65 A	
FIELD("q", "s", "q", "l")		2
FIND_IN_SET("q", "s,q,l")		2
FORMAT(250500.5634, 2)		
CONCAT('Hello', 'World')	HelloWorld	both
TRIM(' HOLA MUNDO ');	HOLA MUNDO	both
LENGTH('HelloWorld')		10 both
INSTR('HelloWorld', 'Wo')		6 both
INSTR('HelloWorld', 'o', 1, 2)		7 no
INSTR('HelloWorld', 'o', -1)		7 no
INSTR('HelloWorld', 'o', -1, 2)		5 no
SUBSTR('HelloWorld', 1, 5)	Hello	both
SUBSTR('HelloWorld', 6)	World	both
SUBSTR('HelloWorld', -3, 1)	r	both
where SUBSTR('Abel', -2, 1)='a'		
SUBSTR(SYSDATE, 8)	'YY' or 'YYYY'	both
LPAD(salary, 10, '*')	*****24000	both
RPAD(salary, 10, '*')	24000*****	both
REPEAT('*', 3)	***	
REPLACE('JACK and JUE', 'J', 'BL')	BLACK and BLUE	Both
INITCAP('UPA')	Upa	No existe
lower('Upa')	upa	both
upper('Upa')	UPA	both
reverse('UPA')	APU	both
strcmp("carlo", "carla")	0 1 -1 respecto al primero	
SUBSTRING_INDEX("upa.edu.mx", ".", 2)	upa.edu	
SUBSTRING_INDEX("upa.edu.mx", ".", -1)	mx	



Funciones Numericas	Resultado	MySQL/Oracle
RAND()	[0 ..1)	
FLOOR(min + RAND() * (max - min + 1))	[min..max]	
ROUND(45.926, 2)	45.93	both
ROUND(45.923,-1)	50	
TRUNCATE(45.926, 2)	45.92	/TRUNC
TRUNCATE(45.923,-1)	40	
CEIL(25.1)	26	
FLOOR(25.1)	25	
MOD(5, 3)	2	both
5 % 3	2	
5 DIV 3;	1	Entera
least(5,3,5,2,8)	2	both
greatest(5,3,5,2,8)	8	both
power(2,3) or pow()	8	both
SQRT(25)	5	
exp(1)	2.7172	both
abs(-5)	5	both
ln(10) or log()	2.3025	both
log(10,100) or log10()	2	both
SIN(45 * PI()/180);	0.7071	
ASIN(0.7071) * 180/PI()	45	
RADIANS(45)	0.7854	
DEGREES(0.7854);	45	
sign(12)	1	both
sign(0)	0	
sign(-12)	-1	

Funciones de Fechas	Resultado
current_date(), sysdate(), now()	
user(), current_user(), session_user()	
LAST_DAY("2017-06-15");	30/06/2017
ADDDATE("2026-02-25", INTERVAL 10 DAY)	07/03/2026
SUBDATE("2017-06-15", INTERVAL 10 DAY)	05/06/2017
DATE("2017-06-15")	Convierte a fecha
DATEDIFF("2017-06-25", "2017-06-15")	Regresa Numero de dias
DATE_FORMAT("2017-06-15", "%Y")	Revisar tabla: 2017
year("2017-06-15") , month(), day()	2017
DAYNAME("2017-06-15")	Thursday
DAYOFWEEK(CURDATE())	1=Sunday. .7=Saturday
WEEKDAY(sysdate())	0=Monday. .6=Sunday
EXTRACT(MONTH FROM "2017-06-15")	both 1..12
MONTHNAME("2017-06-15")	June
STR_TO_DATE("August 10 2017", "%M %d %Y");	10/08/2017

➔

Format	Description
%a	Abbreviated weekday name (Sun to Sat)
%b	Abbreviated month name (Jan to Dec)
%c	Numeric month name (0 to 12)
%D	Day of the month as a numeric value, followed by suffix (1st, 2nd, 3rd, ...)
%d	Day of the month as a numeric value (01 to 31)
%e	Day of the month as a numeric value (0 to 31)
%H	Hour (00 to 23)
%h	Hour (00 to 12)
%i	Minutes (00 to 59)
%j	Day of the year (001 to 366)
%M	Month name in full (January to December)
%m	Month name as a numeric value (00 to 12)
%p	AM or PM
%r	Time in 12 hour AM or PM format (hh:mm:ss AM/PM)
%s	Seconds (00 to 59)
%T	Time in 24 hour format (hh:mm:ss)
%u	Week where Monday is the first day of the week (00 to 53)
%V	Week where Sunday is the first day of the week (01 to 53). Used with %X
%v	Week where Monday is the first day of the week (01 to 53). Used with %x
%W	Weekday name in full (Sunday to Saturday)
%w	Day of the week where Sunday=0 and Saturday=6
%X	Year for the week where Sunday is the first day of the week. Used with %V
%x	Year for the week where Monday is the first day of the week. Used with %v
%Y	Year as a numeric, 4-digit value
%y	Year as a numeric, 2-digit value



```

Folder: Mexico_MySQL
use MEXICO;

update ciudades set capital = 'zacatecas'
where IdCiudad = 32;

alter table ciudades
ADD CONSTRAINT PK_ciudades PRIMARY KEY (IdCiudad);
or
ALTER TABLE ciudades MODIFY IdCiudad INT AUTO_INCREMENT,
ADD CONSTRAINT PK_ciudades PRIMARY KEY (IdCiudad);

alter table ciudades
add constraint check_sexo CHECK( sexo IN (true, false, null));

```

4. S04

Instalacion de DB

MEXICO CLINICA HR PLANS

Advanced Functions or Flow Control

IF = nvl2 oracle CASE IFNULL = nvl oracle NULLIF COALESCE

```

SELECT IF(500<1000, "YES", "NO") answer;

SELECT abr,
       if(sexo=true, "H", if (sexo=false, 'M', 'X')) genero,
       gobernador
from ciudades;

SELECT employee_id, salary, COMMISSION_PCT, if(commission_pct,1,0) from employees;

SELECT case when 500<1000 then "YES" else "NO" end answer;

select abr,
       case when sexo = true then 'H'
            when sexo = false then "M"
            else 'X'
       end genero,
       gobernador
from ciudades;

Select ifnull(null, 0); -- Regresa 0 si la expresión es nula, de lo contrario, regresa su valor

Select salary, ifnull(commission_pct, 0) comision
from employees;

```

- The NULLIF() function compares two expressions and returns NULL if they are equal. Otherwise, the first expression is returned.

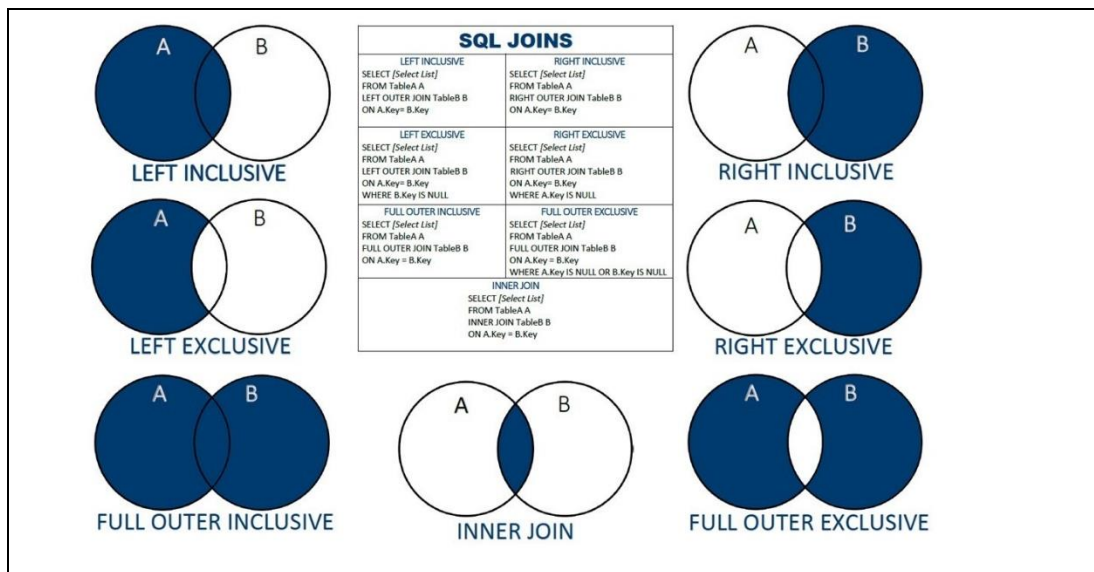
```
SELECT NULLIF("Hello", "world");
SELECT NULLIF("2017-08-25", "2017-08-25");
```

- **COALESCE** Returns the first non-null expression in the list

```
SELECT COALESCE(NULL, 1, 2, 'W3Schools.com');
```

```
Select employee_id, salary, commission_pct,
       COALESCE(salary+commission_pct*salary, salary) sueldo
From employees;
```

5. S05



6. S06

- Left, not in, exists

Subconsultas

```
SELECT fields, ( select : escalar )
FROM table1 inner join ( select : tabla)
WHERE field1 = ( select : escalar)
      Field1 in ( select : tabla)
      Field1 > all (select : tabla)
      Fields < any (select : tabla)
GROUP BY
HAVING function_agrupada >= ( select : escalar )
ORDER BY
```

Promedio

Ciudad con el numero total de municipios

Pacientes con el maximo numero de citas

7. S07

MySQL Shutdown

1. Stop all Services from XAMPP
2. Rename folder mysql/data to mysql/data_old
3. Make a copy of mysql/backup folder and name it as mysql/data
4. Copy all your database folders from mysql/data_old into mysql/data
(except mysql, performance_schema, phpmyadmin and test folders)
5. Copy mysql/data_old/ibdata1 file into mysql/data folder
6. Start MySQL from XAMPP control panel

root:localhost, ya no tiene password
desaparecen todos los usuarios

mysql -> Seleccionar todo -> Reparar la tabla
→

8. S08

Las Funciones operan sobre cada registro.

Los procedimientos tienen varios propósitos: Inserciones, Validaciones, Reportes, Mantenimiento

Funciones: Functions delimiter \$\$ create function fnX (dato int) returns char(10) deterministic begin declare leyenda char(10); . . . return leyenda; end; \$\$ select fnX(10) from dual;	Procedimientos: Stored Procedures DELIMITER \$\$ CREATE PROCEDURE spValidarEmail(email VARCHAR(100)) BEGIN declare leyenda char(10); . . . END; \$\$ call spValidarEmail("carlos@gmail.com");
--	---

Using Metacharacters with Regular Expressions <table><tr><th>Syntax</th><th>Description</th></tr><tr><td>.</td><td>Matches any character in the supported character set, except NULL.</td></tr><tr><td>+</td><td>Matches one or more occurrences</td></tr><tr><td>?</td><td>Matches zero or one occurrence</td></tr><tr><td>*</td><td>Matches zero or more occurrences of the preceding subexpression</td></tr><tr><td>{m}</td><td>Matches exactly <i>m</i> occurrences of the preceding expression</td></tr><tr><td>{m, }</td><td>Matches at least <i>m</i> occurrences of the preceding subexpression</td></tr><tr><td>{m, n}</td><td>Matches at least <i>m</i>, but not more than <i>n</i>, occurrences of the preceding subexpression</td></tr><tr><td>[...]</td><td>Matches any single character in the list within the brackets</td></tr><tr><td> </td><td>Matches one of the alternatives</td></tr><tr><td>(...)</td><td>Treats the enclosed expression within the parentheses as a unit. The subexpression can be a string of literals or a complex expression containing operators.</td></tr><tr><td>^</td><td>Matches the beginning of a string</td></tr><tr><td>\$</td><td>Matches the end of a string</td></tr><tr><td>\</td><td>Treats the subsequent metacharacter in the expression as a literal</td></tr><tr><td>\n</td><td>Matches the <i>n</i>th (1-9) preceding subexpression of whatever is grouped within parentheses. The parentheses cause an expression to be remembered; a backreference refers to it.</td></tr><tr><td>\d</td><td>A digit character</td></tr><tr><td>[:class:]</td><td>Matches any character belonging to the specified POSIX character class</td></tr><tr><td>[^:class:]</td><td>Matches any single character <i>not</i> in the list within the brackets</td></tr></table>	Syntax	Description	.	Matches any character in the supported character set, except NULL.	+	Matches one or more occurrences	?	Matches zero or one occurrence	*	Matches zero or more occurrences of the preceding subexpression	{m}	Matches exactly <i>m</i> occurrences of the preceding expression	{m, }	Matches at least <i>m</i> occurrences of the preceding subexpression	{m, n}	Matches at least <i>m</i> , but not more than <i>n</i> , occurrences of the preceding subexpression	[...]	Matches any single character in the list within the brackets		Matches one of the alternatives	(...)	Treats the enclosed expression within the parentheses as a unit. The subexpression can be a string of literals or a complex expression containing operators.	^	Matches the beginning of a string	\$	Matches the end of a string	\	Treats the subsequent metacharacter in the expression as a literal	\n	Matches the <i>n</i> th (1-9) preceding subexpression of whatever is grouped within parentheses. The parentheses cause an expression to be remembered; a backreference refers to it.	\d	A digit character	[:class:]	Matches any character belonging to the specified POSIX character class	[^:class:]	Matches any single character <i>not</i> in the list within the brackets	 delimiter \$\$ CREATE FUNCTION fnEspacios(cadena VARCHAR(255)) RETURNS VARCHAR(255) DETERMINISTIC BEGIN DECLARE r VARCHAR(255); -- Reemplaza múltiples espacios por uno solo SET r = TRIM(REGEXP_REPLACE(cadena, '\\s+', ' ')); RETURN resultado; END; \$\$ select fnEspacios(" San Luis Potosi "); San Luis Potosi
Syntax	Description																																				
.	Matches any character in the supported character set, except NULL.																																				
+	Matches one or more occurrences																																				
?	Matches zero or one occurrence																																				
*	Matches zero or more occurrences of the preceding subexpression																																				
{m}	Matches exactly <i>m</i> occurrences of the preceding expression																																				
{m, }	Matches at least <i>m</i> occurrences of the preceding subexpression																																				
{m, n}	Matches at least <i>m</i> , but not more than <i>n</i> , occurrences of the preceding subexpression																																				
[...]	Matches any single character in the list within the brackets																																				
	Matches one of the alternatives																																				
(...)	Treats the enclosed expression within the parentheses as a unit. The subexpression can be a string of literals or a complex expression containing operators.																																				
^	Matches the beginning of a string																																				
\$	Matches the end of a string																																				
\	Treats the subsequent metacharacter in the expression as a literal																																				
\n	Matches the <i>n</i> th (1-9) preceding subexpression of whatever is grouped within parentheses. The parentheses cause an expression to be remembered; a backreference refers to it.																																				
\d	A digit character																																				
[:class:]	Matches any character belonging to the specified POSIX character class																																				
[^:class:]	Matches any single character <i>not</i> in the list within the brackets																																				



9. S09

```
DELIMITER $$
CREATE PROCEDURE spCdPartido(in vPartido VARCHAR(50) )
BEGIN
    declare leyenda char(10);

    select c.IdCiudad, abr
    from ciudades c join gobernadores g on c.idciudad = g.idciudad
    where g.partido = vPartido;
END;
$$

call spCdPartido("pAn");
```

REGEXP `'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'`

<pre>DELIMITER \$\$ CREATE FUNCTION fnIsEmail (correo VARCHAR(100)) RETURNS BOOLEAN DETERMINISTIC BEGIN IF correo REGEXP '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\$' THEN RETURN TRUE; ELSE RETURN FALSE; END IF; END; \$\$ SELECT fnIsEmail('carlos@example.com') isEmail;</pre>	<pre>DELIMITER \$\$ CREATE PROCEDURE splsEmail (email VARCHAR(100)) BEGIN IF email REGEXP '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\$' THEN SELECT 'email válido.' AS mensaje; ELSE SELECT 'Formato de correo inválido.' AS mensaje; END IF; END; \$\$ call splsEmail("carlos@gmail.com");</pre>
---	---